

```
!pip install opencv-contrib-python scikit-image
```

```
Requirement already satisfied: opencv-contrib-python in /usr/local/lib/python3.13/dist-packages (4.14.0.94)
Requirement already satisfied: scikit-image in /usr/local/lib/python3.13/dist-packages (0.25.2)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.13/dist-packages (from opencv-contrib-python) (2.1.3)
Requirement already satisfied: scipy>=1.11.4 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (1.16.3)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (3.6.1)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (2.37.4)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (2026.8.23)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (26.3)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (0.5)
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import exposure

print("All libraries loaded successfully!")
```

All libraries loaded successfully!

```
from google.colab import files
```

```
uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
filename = list(uploaded.keys())[0]

img = cv2.imread(filename)

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

print("Image loaded successfully!")
print("Image size:", img.shape)
```

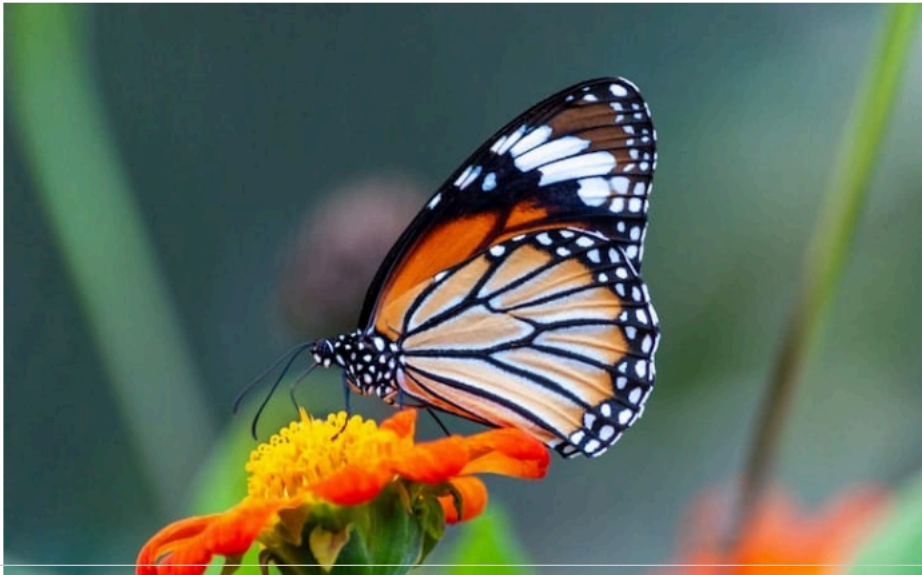
Image loaded successfully!
Image size: (491, 740, 3)

```
plt.figure(figsize=(10, 6))

plt.imshow(img_rgb)
plt.title("Original Butterfly Image")
plt.axis("off")

plt.show()
```

Original Butterfly Image



```
sift = cv2.SIFT_create()

keypoints, descriptors = sift.detectAndCompute(gray, None)

print("Number of SIFT Keypoints:", len(keypoints))
print("Descriptor Shape:", descriptors.shape)
```

```
Number of SIFT Keypoints: 1138
Descriptor Shape: (1138, 128)
```

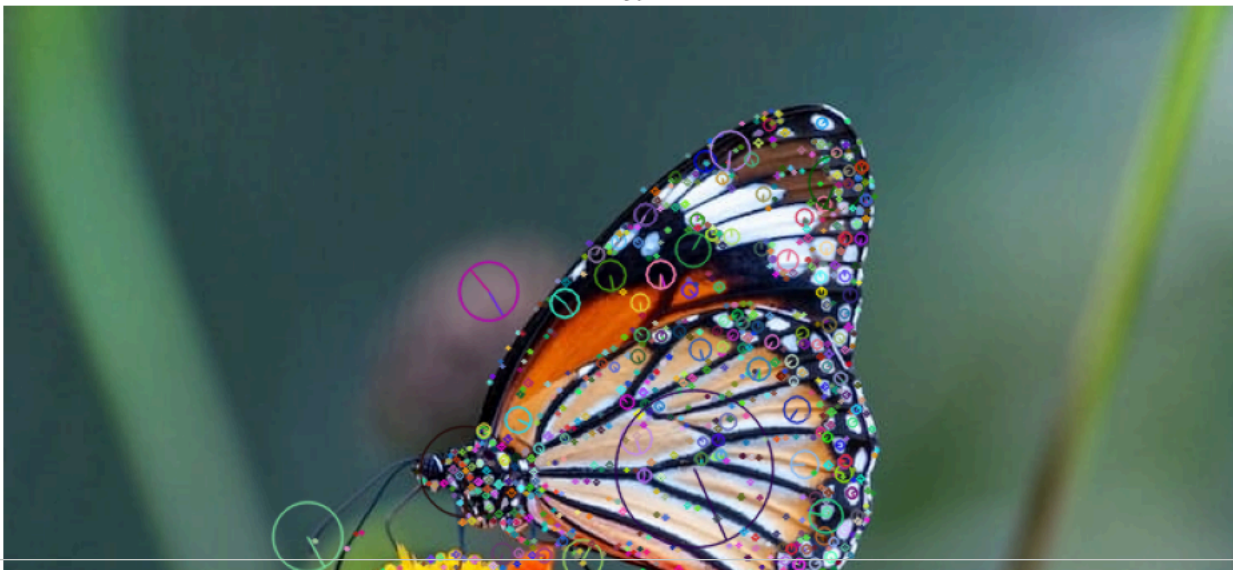
```
sift_image = cv2.drawKeypoints(
    img_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(12, 8))

plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis("off")

plt.show()
```

SIFT Keypoints



```
hog_features, hog_image = hog(
    gray,
```

```

orientations=9,
pixels_per_cell=(8, 8),
cells_per_block=(2, 2),
block_norm='L2-Hys',
visualize=True
)

print("HOG Feature Extraction Completed!")
print("HOG Feature Vector Length:", len(hog_features))

```

HOG Feature Extraction Completed!
HOG Feature Vector Length: 196560

```

hog_image_rescaled = exposure.rescale_intensity(
    hog_image,
    in_range=(0, 10)
)

plt.figure(figsize=(14, 6))

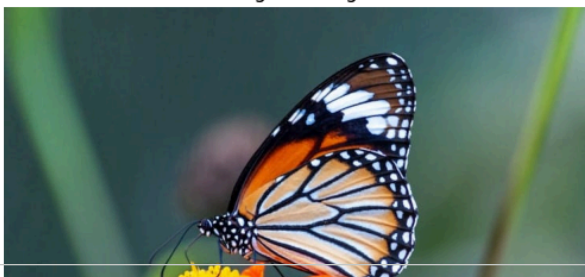
plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_image_rescaled, cmap="gray")
plt.title("HOG Visualization")
plt.axis("off")

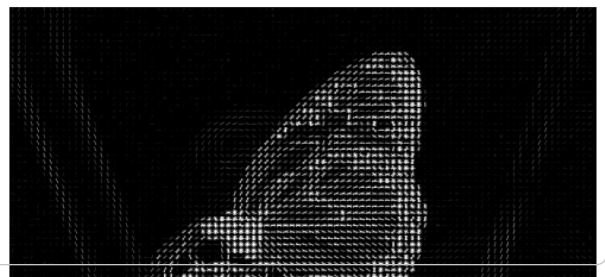
plt.show()

```

Original Image



HOG Visualization



```

img2 = cv2.resize(
    img,
    None,
    fx=5.2,
    fy=5.2
)

gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

print("Second image created!")

```

Second image created!

```

img2 = cv2.resize(
    img,
    None,
    fx=1.2,
    fy=1.2
)

gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

img2_rgb = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)

print("Second image created!")

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")

```

```
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(img2_rgb)
plt.title("Resized Image (1.2x)")
plt.axis("off")

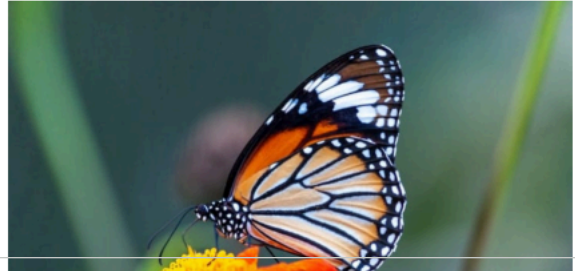
plt.show()
```

Second image created!

Original Image



Resized Image (1.2x)



```
kp1, des1 = sift.detectAndCompute(gray, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

print("Original Image Keypoints:", len(kp1))
print("Second Image Keypoints:", len(kp2))
```

```
Original Image Keypoints: 1138
Second Image Keypoints: 1429
```

```
bf = cv2.BFMatcher()

matches = bf.knnMatch(des1, des2, k=2)

good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Total Matches:", len(matches))
print("Good Matches:", len(good_matches))
```

```
Total Matches: 1138
Good Matches: 685
```

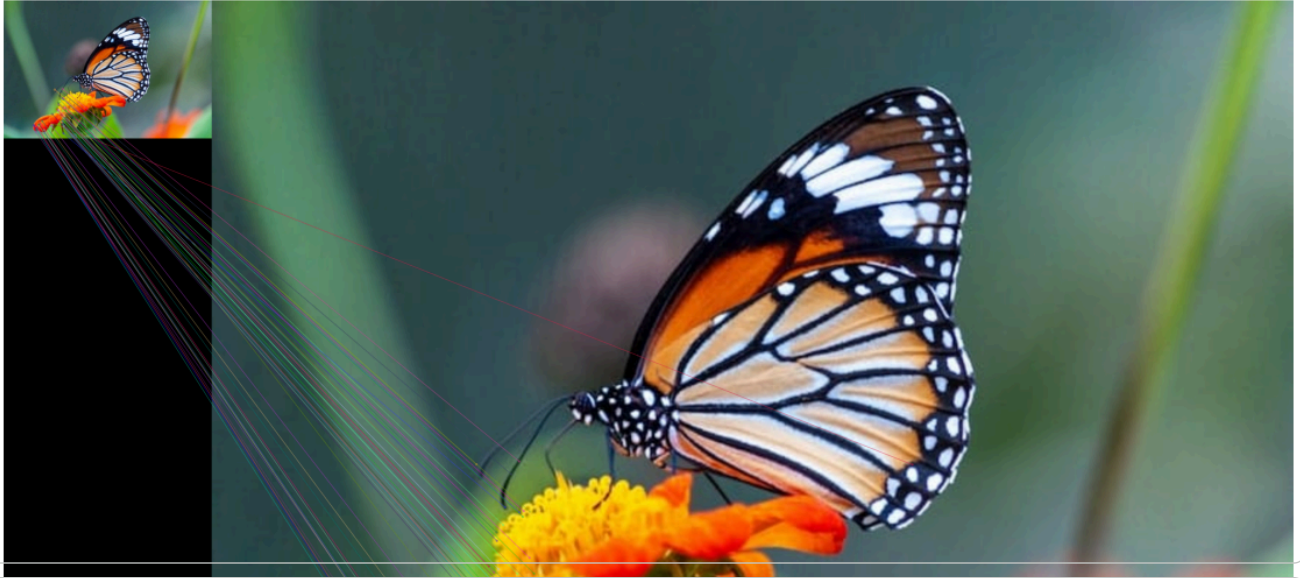
```
matched_image = cv2.drawMatches(
    img,
    kp1,
    img2,
    kp2,
    good_matches[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(18, 8))

plt.imshow(cv2.cvtColor(matched_image, cv2.COLOR_BGR2RGB))
plt.title("SIFT Feature Matching")
plt.axis("off")

plt.show()
```

SIFT Feature Matching



```
plt.figure(figsize=(10, 6))  
plt.imshow(img_rgb)  
plt.title("Original Colour Image")  
plt.axis("off")  
plt.show()
```

Original Colour Image

